

Automatic Test Cases Generation Method by Structuring Specification Documents Using Seq2Seq

1st Yuki Shimizu
Graduate School of Engineering
Nihon University
Koriyama, Japan
ceyu24010@g.nihon-u.ac.jp

2nd Kiyoshi Ueda
College of Engineering
Nihon University
Koriyama, Japan
ueda.kiyoshi@nihon-u.ac.jp

Abstract—We aim to automate the development of communication software. By leveraging machine learning, we propose a method for automatically generating system testing test cases from requirement specification documents. In this research, we focus on developing a method to generate test cases by structuring requirement specification sentences using a deep learning Seq2Seq model with an Attention mechanism. Our goal is to further enhance the accuracy of this approach. To evaluate the effectiveness and characteristics of the proposed method, we conducted experiments using requirement specifications of communication system software and various IT system requirement specification documents.

Index Terms—test cases generation, machine learning, LSTM, Seq2Seq, attention.

I. INTRODUCTION

In the development of communication software, the waterfall model is widely used as the mainstream approach. Fig. 1 illustrates the concept of automatically generating test cases within the V-model framework. The lower-level processes of the V-model have already been automated, but it is difficult to automate the extraction of system test cases from the natural language requirement specifications in the upper-level processes. In the development of high-quality, large-scale communication software, improving efficiency and automating the extraction of system test cases from specifications has been a persistent challenge. To address this, we have been investigating a method for automatically generating system test cases from requirement specifications using machine learning [1]. To further enhance the accuracy of test cases generated with machine learning, we aim to develop a method for automatic test case generation by structuring requirement specifications using deep learning.

II. PREVIOUS RESEARCH

A. Automatic Test Case Generation Method

The method for automatically generating test cases is divided into four phases. The process flow of this method is illustrated in Fig. 2.

1) *Phase of Structuring the Requirements Specification (Tagging)*: First, experts assign tags to parts of the requirement specification corresponding to the test cases. These tags indicate input/output information, checkpoints for each function, and other details, thereby structuring the requirement

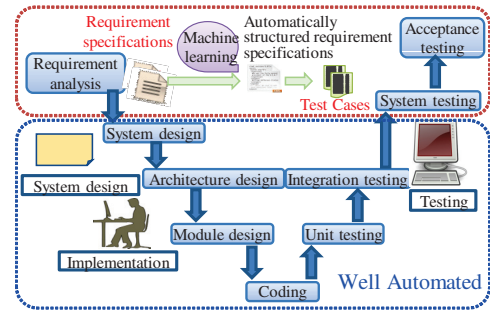


Fig. 1. Image of automatic generation of test cases in V-model.

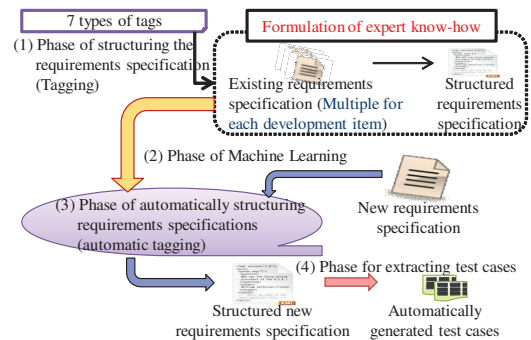


Fig. 2. Automatic test cases generation method.

specification (formulation of expert know-how). For example, in Fig. 3, the section describing the trigger for the action, "when the caller dials the time signal service number (117)," is structured by enclosing it within the <Input>tag.

2) *Phase of Machine Learning*: The structured requirement specification is divided into morphemes (the smallest units of words that have meaning) using morphological analysis software (MeCab) [2], and parts-of-speech are assigned to create the training data. By dividing into morphemes, the tag information assigned to the sentence is assigned to each morpheme. Fig. 4 shows an example of data processing according to the flow of the automatic test cases generation method. For example, the part enclosed in the <Input_Cnd>tag "AC

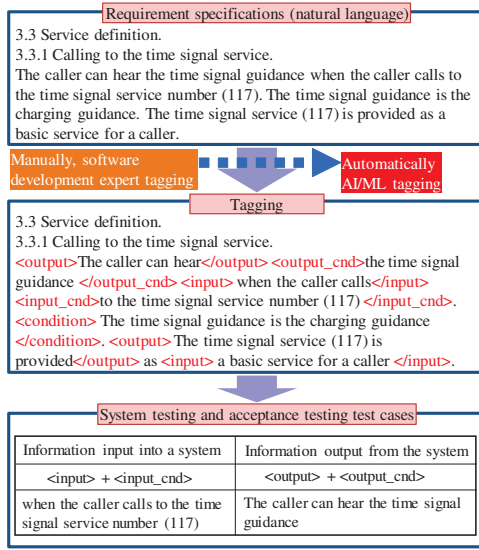


Fig. 3. Tagging for structuring requirements specification.

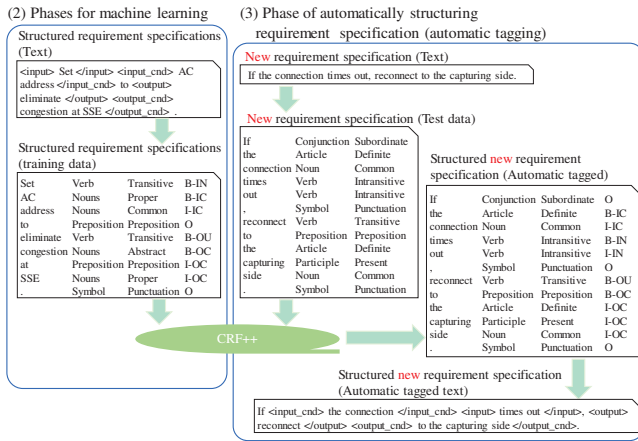


Fig. 4. Automatically structuring data processing.

address” is broken down into the morphemes “AC” and “address” by morphological analysis, and B-IC is assigned to “AC” and I-IC is assigned to “address”. The training data with tags assigned to each morpheme is then used to train the machine learning tool CRF++ [3]. At this time, a model file is generated that becomes the learning model for machine learning.

3) *Phase of Automatically Structuring Requirements Specifications (Automatic Tagging)*: The new requirement specification is analyzed using morphological analysis. When the new requirement specification is input into CRF++, which has been trained using the model file generated in the previous phase, a new requirement specification with tags for each word is created. This is called automatic structuring.

4) *Phase for Extracting Test Cases*: The test cases are automatically extracted by a program that extracts the parts

with tags indicating input and output information, etc. from the new requirement specifications that have been structured in the automatic structuring phase.

5) *Evaluation Method for Automatic Test Item Generation Method*: The accuracy of the automatic structuring of the requirements specification is calculated by calculating the *Precision* and *Recall* for each tag assigned to each word in the requirements specification using the formulas shown in equations (1 and 2), and then calculating the *F-value* (harmonic mean) shown in equation (3), and calculating this as the accuracy.

$$Precision = \frac{TP}{(TP + FP)} \quad (1)$$

$$Recall = \frac{TP}{(TP + FN)} \quad (2)$$

$$F-value = \frac{2 \times Precision \times Recall}{(Precision + Recall)} \quad (3)$$

TP: Number of words for which the correct tag manually added by the expert matched the tag predicted by the machine learning system.

[Ex.] Number of words for which the correct tag was I-IN and the predicted tag was I-IN.

FP: Number of words for which the tag predicted by the machine learning system did not match the correct tag manually added by the expert.

[Ex.] The number of words for which the predicted tag was I-IN, but the correct answer was a tag other than I-IN.

FN: The number of words for which the machine learning system's predicted tag did not match the correct answer tag manually added by the expert.

[Ex.] The number of words for which the correct answer tag was I-IN, but the predicted tag was a tag other than I-IN.

B. Learning and Automatic Structuring Using Deep Learning

We have been using CRF for machine learning to learn the requirements specification documents and improve the accuracy of automatic structuring. We would like to further improve the accuracy of automatic structuring of the requirements specification documents.

Therefore, we thought that we could achieve a high level of accuracy by using deep learning, which has been attracting attention in recent years, to learn and perform automatic structuring. We used deep learning, which allows the computer to find features and learn without specifying features, to perform automatic structuring.

1) *Recurrent Neural Network (RNN)*: RNN is a type of neural network (NN) [4] that is based on the structure of the human nervous system, and consists of a multilayer network with an input layer, multiple hidden layers (intermediate layers) and an output layer. RNN is a NN model designed to recognize patterns in sequence data such as numerical time series data. RNN can remember information from previous words and use this information to predict new events.

C. Long Short-Term Memory (LSTM)

RNNs are unable to learn long-term dependencies due to the vanishing gradient problem. For this reason, LSTM [4] was designed as an extension of RNNs to avoid the problem of long-term dependencies. In LSTM, three gates (forget gate, input gate, and output gate) are created in the middle layer of the RNN, and long-term dependencies can be learned by manipulating long-term and short-term memories.

D. Optimizer and Initial Learning Rate

The optimizer (optimization algorithm) is an algorithm that seeks to minimize the difference (loss) between the output prediction value and the correct value as efficiently as possible. We used Adam [5], which adjusts the learning rate using the moving average of the gradient and the moving average of the square of the gradient. If the learning rate is too high, learning will be fast but there is a risk of exceeding the optimal solution, and if it is too low, there is a risk of learning being slow. We adjusted the initial learning rate and set the optimal value by comparing the results of the experiment.

1) *Automatic Specification Structuring Method Using LSTM*: We used LSTM to train and automatically structure 20 documents (4,983 sentences in total) of requirement specifications for session control servers, which are important servers in communication networks, and 14 documents (548 sentences in total) of publicly available and obtainable IT system specifications. We calculated the correct response rate and compared it with the accuracy when using CRF. When using the session control server requirement specification, the accuracy was around 30%, which was not as high as the accuracy of CRF, but it still produced some results. We also confirmed the general effectiveness by changing the specification that was used as the teacher data to the various IT system specifications, and achieving an accuracy of around 50%.

2) *Automatic Specification Structuring Method Using the Encoder-Decoder Model (Seq2Seq Model)*: There is a Seq2Seq model that uses two LSTMs, one to convert time-series data into feature vectors and the other to decode feature vectors into time-series data. We changed the LSTM model to a Seq2Seq model and trained and predicted it. However, the accuracy was lower than when using LSTM [5]. We thought this was because the two LSTMs converted the time-series information for each word into fixed-length vectors and passed them on to each other.

E. Issues in Previous Research

Although the LSTM model achieved a certain level of accuracy, it did not reach the accuracy of CRF, and the Seq2Seq model achieved even lower accuracy. Therefore, it is necessary to create an even better model.

III. PROPOSED METHOD USING ATTENTION-BASED SEQ2SEQ MODEL

Attention is a mechanism for determining which parts of the sentence data are important and which words should be paid

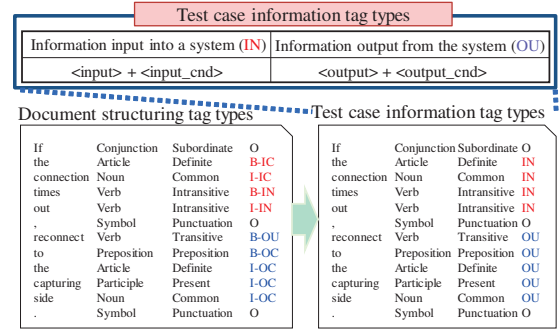


Fig. 5. Conversion to test case information tag types.

attention to. In the Seq2Seq model, the time series information is converted into fixed-length vectors and passed between the encoder and decoder, but Attention converts it into a contextual vector that takes into account which words should be paid attention to, so it is possible to consider the relationships between words. The Transformer, which is the basis for the natural language model in the generative AI, uses a model that incorporates Attention into the Seq2Seq model. We will apply this model for automatic test cases generation (hereafter, Attention Seq2Seq model method).

IV. EVALUATION

A. Evaluation Perspectives

We will evaluate the accuracy using 20 sets of session control server requirement specifications (Evaluation 1). We will compare the accuracy with that of the CRF in Section II-A and the LSTM model in Section II-D1, and check whether the accuracy is improved. We will conduct experiments to find the optimal value for the number of epochs with an initial learning rate of 0.001 and 50 epochs (Attention-50), other initial learning rate of 0.0001 and 100 epochs (Attention-100). The proposed method uses a batch size of 20.

We evaluate the accuracy using not only the session control server's requirement specification, but also 14 various IT system specifications for learning and prediction (Evaluation 2). We compare the accuracy of the CRF in Section II-A and the LSTM model in Section II-D1 when using the various IT system specifications, and confirm the versatility of the proposed method. We perform experiments with an initial learning rate of 0.001 and 50 epochs (Attention-50), other initial learning rate of 0.0001 and 100 epochs (Attention-100). The proposed method uses a batch size of 20.

B. Evaluation Conditions

As evaluation conditions, we cut out particles, symbols, and auxiliary verbs and compare the accuracy when B-IN and I-IN, B-IC and I-IC are replaced with IN, and B-OU and I-OU, B-OC and I-OC are replaced with OU, as shown in Fig. 5. After tag grouping, the accuracy is expressed in terms of test items.

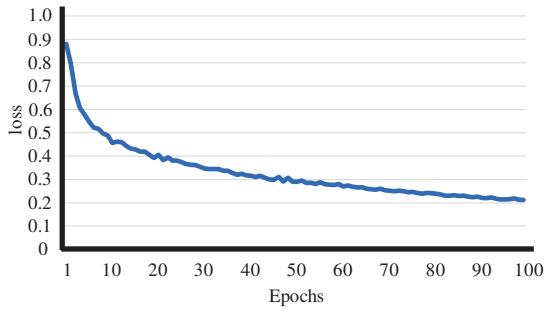


Fig. 6. Loss value for Epochs in the proposed method.

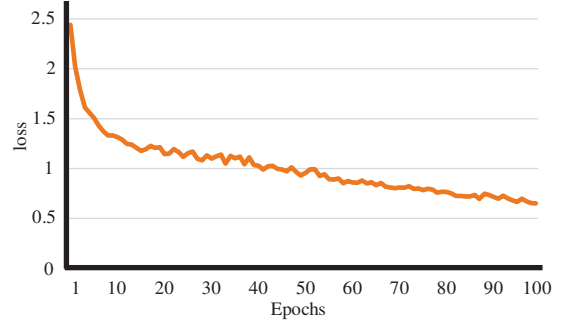


Fig. 8. Loss value for Epochs for various IT system requirement specifications.

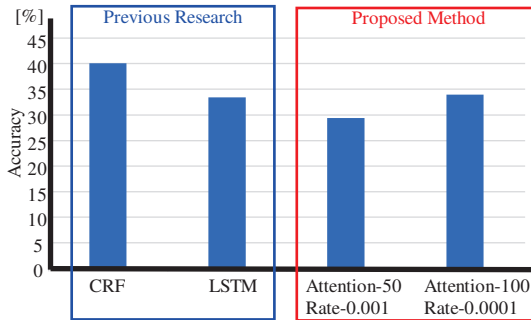


Fig. 7. Accuracy of each machine learning method.

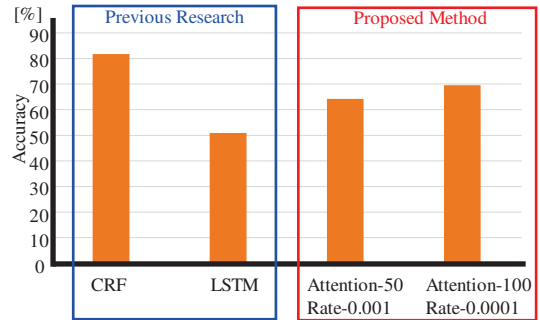


Fig. 9. Accuracy of each machine learning methods for various IT system requirement specifications.

C. Evaluation Results

1) *Accuracy in the Session Control Server Requirement Specification (Evaluation 1)*: The loss values when using the session control server requirement specification are shown in Fig. 6, and the accuracy is shown in Fig. 7.

Although the loss values were normal, the accuracy for both Attention-50 and Attention-100 was lower than the accuracy of the LSTM in the previous study. In addition, we confirmed that Attention-100 had a higher accuracy. We measured accuracy over 100 epochs, but accuracy decreased, so the optimal number of Epochs was around 100. The analysis of the loss value showed a tendency toward overfitting.

In Attention-100, the accuracy of LSTM was almost the same as that of CRF, although it was not as accurate as CRF. This is probably because the program has not been optimized for learning. A t-test is needed for an accurate evaluation.

2) *Accuracy in IT System Requirement Specifications (Evaluation 2)*: The loss values in the various IT system specifications are shown in Fig. 8, and the accuracy is shown in Fig. 9. The results for Attention-50 are for 50 epochs, and the results for Attention-100 are for 100 epochs.

The loss values are also normal. Both Attention-50 and Attention-100 achieve a slightly higher accuracy than the accuracy of LSTM in previous studies. The accuracy of Attention-100 is about 70%, which is not as high as the accuracy of CRF, but it is still a reasonable accuracy.

V. CONCLUSION

We proposed a method for automatically generating test items using an Attention-based Seq2Seq model. While the method achieved high accuracy across a variety of IT system specifications, its performance was lower when session control server specification documents were used as training data, compared to previous studies.

The significant variation in accuracy depending on the specification document highlights the need to analyze the content of these documents. Specifically, parts of the specification that are unsuitable for training should be identified and excluded. Furthermore, to enhance accuracy, it is essential to explore methods incorporating Large Language Models (LLMs).

REFERENCES

- [1] Y. Fujita, K. Ueda, "A Method for Selecting Training Data Using Doc2Vec for Automatic Test Cases Generation," Proc. of ICCE2024, IEEE, Jan. 2024.
- [2] MeCab: Yet Another Part-of-Speech and Morphological Analyzer, <http://taku910.github.io/mecab/>.
- [3] CRF++: Yet Another CRF Toolkit, <http://taku910.github.io/crfpp/#source>.
- [4] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, pp.1735–1780, Nov. 1997.
- [5] Y. Shimizu, K. Ueda, "Automatic Test Cases Generation Method by Structuring Requirement Specification Using LSTM," Proc. of IEEE Int. Conf. on Consumer Electronics - Taiwan, IEEE, July 2024.